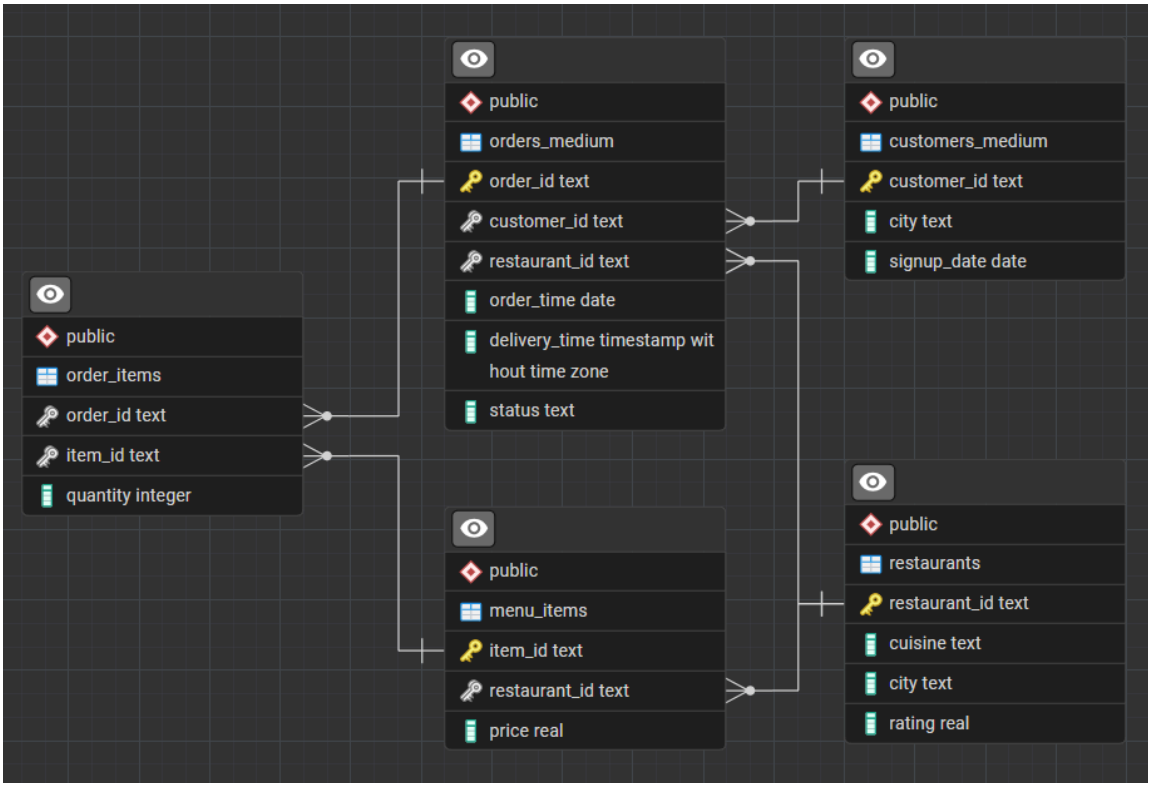
Food Delivery: Queries & Insights

Queries

-- ERD



-- Q1 Number of customers in each city

```
SELECT city, COUNT (customer_id)
FROM customers_medium
GROUP BY 1
ORDER BY 2 DESC;
```

	city text	count bigint
1	Bristol	270
2	Leeds	252
3	Liverpool	247
4	London	246
5	Birmingham	245
6	Manchester	240

-- Q2 Number of restaurants under each cuisine

```sql
SELECT cuisine, COUNT(restaurant_id)
FROM restaurants
GROUP BY 1
ORDER BY 2 DESC;
```

	cuisine text	count bigint
1	Thai	26
2	Indian	22
3	Italian	20
4	Mexican	20
5	American	19
6	Chinese	13

-- Q3 Top rated restaurants

```sql
SELECT
    restaurant_id, rating,
    CASE
        WHEN rating = 5 THEN 'Excellent'
        WHEN rating >= 4 THEN 'Good'
        WHEN rating >= 3 THEN 'Average'
        ELSE 'Poor'
    END AS rating_category
FROM restaurants
ORDER BY 2 DESC;
```

	restaurant_id [PK] text	rating real	rating_category text
1	R020	5	Excellent
2	R030	5	Excellent
3	R115	5	Excellent
4	R093	4.9	Good
5	R042	4.9	Good
6	R083	4.9	Good
7	R086	4.9	Good

-- Q4 Number of customers that ordered from each restaurant

```
SELECT
    r.restaurant_id,
    COUNT(DISTINCT o.customer_id) AS num_of_customers
FROM restaurants r
LEFT JOIN orders_medium o
ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id
ORDER BY r.restaurant_id;
```

	restaurant_id [PK] text	num_of_customers bigint
1	R001	40
2	R002	29
3	R003	46
4	R004	34
5	R005	32
6	R006	37
7	R007	36
8	R008	30
9	R009	49
10	R010	40

-- Q5 Average delivery time per restaurant

```
SELECT
    r.restaurant_id,
    ROUND (AVG(EXTRACT(EPOCH FROM (o.delivery_time -
o.order_time)) / 60), 2) AS avg_delivery_minutes
FROM orders_medium o
JOIN restaurants r
ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id
ORDER BY avg_delivery_minutes;
```

	restaurant_id [PK] text	avg_delivery_minutes numeric
1	R010	45.98
2	R115	47.12
3	R084	48.23
4	R044	48.53
5	R120	49.53
6	R018	49.64
7	R111	49.72
8	R116	49.81
9	R015	50.00
10	R095	50.34

-- Q6 Top-Grossing Restaurants by Cuisine

```
SELECT
    r.restaurant_id, r.cuisine,
    ROUND(SUM(i.quantity * m.price)::numeric, 2) AS revenue
FROM restaurants r
JOIN orders_medium o
ON r.restaurant_id = o.restaurant_id
JOIN order_items i
ON o.order_id = i.order_id
JOIN menu_items m
ON i.item_id = m.item_id
GROUP BY r.restaurant_id, r.cuisine
ORDER BY revenue DESC;
```

	restaurant_id [PK] text	cuisine text	revenue numeric
1	R061	Thai	6232.12
2	R021	Italian	6108.38
3	R032	American	5671.31
4	R011	Italian	5664.40
5	R046	Mexican	5596.77
6	R060	Indian	5364.45
7	R067	Italian	5260.59
8	R088	American	5250.62
9	R055	American	5141.67
10	R026	Indian	5137.67

-- Q7 Ranking Restaurants by Orders

```
SELECT
    restaurant_id,
    COUNT(*) AS total_orders,
    DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS rank
FROM orders_medium
GROUP BY restaurant_id;
```

	restaurant_id text	total_orders bigint	rank bigint
1	R021	57	1
2	R060	56	2
3	R022	55	3
4	R038	55	3
5	R088	54	4
6	R096	54	4
7	R046	53	5
8	R032	53	5
9	R074	53	5
10	R061	51	6
11	R078	51	6

-- Q8 Customer Orders and Spending Summary

```
SELECT
    c.customer_id,
    ROUND(COALESCE(SUM(o1.quantity * m.price), 0)::numeric, 2) AS
total_spent,
    ROUND(
        CASE
            WHEN COUNT(DISTINCT o.order_id) = 0 THEN 0
            ELSE SUM(o1.quantity * m.price)::numeric /
COUNT(DISTINCT o.order_id)
        END,
        2
    ) AS avg_per_order,
    COUNT(DISTINCT o.order_id) AS num_of_orders
FROM customers_medium c
LEFT JOIN orders_medium o
    ON c.customer_id = o.customer_id
LEFT JOIN order_items o1
    ON o.order_id = o1.order_id
LEFT JOIN menu_items m
    ON m.item_id = o1.item_id
GROUP BY c.customer_id;
ORDER BY total_spent DESC;
```

	customer_id [PK] text	total_spent numeric	avg_per_order numeric	num_of_orders bigint
1	C0001	606.47	151.62	4
2	C0002	422.18	84.44	5
3	C0003	454.91	90.98	5
4	C0004	233.12	233.12	1
5	C0005	231.84	77.28	3
6	C0006	570.65	95.11	6
7	C0007	910.67	113.83	8
8	C0008	144.29	48.10	3
9	C0009	0.00	0.00	0
10	C0010	134.60	67.30	2
11	C0011	521.80	86.97	6
12	C0012	230.22	76.74	3
13	C0013	153.68	76.84	2

-- Q9 Top-Selling Menu Item per Restaurant

```sql
SELECT
    restaurant_id,
    item_id AS top_item,
    total_quantity_ordered
FROM (
    SELECT
        m.restaurant_id,
        m.item_id,
        SUM(o.quantity) AS total_quantity_ordered,
        RANK() OVER (
            PARTITION BY m.restaurant_id
            ORDER BY SUM(o.quantity) DESC
        ) AS rank
    FROM menu_items m
    JOIN order_items o
    ON m.item_id = o.item_id
    GROUP BY m.item_id, m.restaurant_id
) sub
WHERE rank = 1
ORDER BY restaurant_id;
```

	restaurant_id text	top_item text	total_quantity_ordered bigint
1	R001	M0265	53
2	R002	M0143	64
3	R003	M0005	47
4	R004	M0305	55
5	R005	M0072	51
6	R006	M0247	68
7	R007	M0135	63
8	R008	M0270	65
9	R009	M0223	66
10	R010	M0279	55

-- Q10 The revenue generated each month in each year

```
SELECT
    TO_CHAR(signup_date, 'MM') AS signup_month,
    COUNT(customer_id) AS num_customers,
    COUNT(customer_id) - LAG(COUNT(customer_id)) OVER (ORDER BY
TO_CHAR(signup_date, 'MM')) AS diff_from_prev_month
FROM customers_medium
GROUP BY signup_month
ORDER BY signup_month;
```

	signup_month text	num_customers bigint	diff_from_prev_month bigint
1	01	110	[null]
2	02	110	0
3	03	146	36
4	04	141	-5
5	05	127	-14
6	06	112	-15
7	07	138	26
8	08	131	-7
9	09	136	5
10	10	142	6
11	11	150	8
12	12	57	-93